

Практичне заняття № 2

Тема: Розробка технічного завдання проекту

Мета: Навчитися оформлювати результати аналізу вимог у вигляді формального Технічного завдання (ТЗ), обираючи структуру документа — за ГОСТ 34.602-89 або за міжнародним стандартом SRS (IEEE 830 / ISO 29148) — залежно від типу проекту та вимог замовника.

Обраний стандарт: SRS

Склад команди: Баль М. та Сівук М.

Завдання 1. Вибір і обґрунтування стандарту

Критерії	Наш проект
Замовник — державна чи комерційна організація	Комерційна мережа кав'ярень CoffeeGo
Чи потрібна юридична сила документа (тендер, держзакупівля)	Ні, документ призначений для внутрішньої розробки
Методологія розробки (Waterfall / Agile)	Agile
Чи є вимога до трасованості (traceability) вимог до тестів	Так
Аудиторія документа (лише команда чи зовнішній замовник/аудитор)	Команда розробки та замовник

2. Для цього проекту краще використовувати SRS (IEEE 830 / ISO 29148). CoffeeGo — це комерційний мобільний застосунок, який розробляється за Agile,

тому вимоги можуть уточнюватися під час роботи. У SRS зручно пов'язувати вимоги з User Stories, Use Case та тестами. Це допоможе команді швидше працювати й встигнути випустити застосунок до 1 грудня.

Завдання 2. Загальні відомості та призначення

1. У цьому документі описані вимоги до мобільного застосунку CoffeeGo. Він потрібен для того, щоб команда розробки й замовник однаково розуміли, які функції має виконувати система.

У першій версії користувачі зможуть замовляти каву заздалегідь, оплачувати її картою, Apple Pay або Google Pay, отримувати Push-сповіщення про готовність замовлення та накопичувати бонусні бали. Також застосунок працюватиме на iOS і Android, буде інтегрований із наявною POS-системою та матиме окремі кабінети для менеджера і баристи.

У документі використовуються такі скорочення: SRS — документ із вимогами до системи, POS — касова система, MVP — перша версія продукту, FR — функціональна вимога, NFR — нефункціональна вимога, UC — Use Case, а US — User Story.

Під час підготовки документа використано стандарт IEEE 830 / ISO 29148, результати Практичного заняття №1 та вимоги замовника проєкту CoffeeGo.

2. CoffeeGo — це новий мобільний застосунок, який працюватиме разом із наявною касовою системою кав'ярні. Коли клієнт оформить замовлення, воно автоматично з'явиться у баристи.

Основні функції системи:

- вибрати одну з трьох кав'ярень;
- переглянути меню;
- оформити замовлення;
- оплатити його онлайн;
- отримувати бонуси;
- отримувати сповіщення про готовність;
- менеджеру переглядати всі замовлення;
- баристі змінювати статус замовлення.

Основними користувачами системи будуть клієнти, баристи та менеджери.

Клієнтам потрібен швидкий спосіб оформити й оплатити замовлення без черги.

Баристам необхідно бачити нові онлайн-замовлення та змінювати їхній статус під час приготування. Менеджери використовуватимуть систему для перегляду всіх замовлень і контролю роботи трьох кав'ярень.

Передбачається, що наявна POS-система підтримує інтеграцію із застосунком, користувачі мають доступ до інтернету, а система підтримує оплату через Apple Pay, Google Pay і банківські картки. Також важливою умовою є завершення розробки та запуск застосунку до 1 грудня.

Завдання 3. Функціональні вимоги

ID	Вимога	Джерело	MoSCoW	Критерій прийнятності	Трасованість
FR-001	Користувач може створити обліковий запис	US-1	Must	Реєстрація завершується за ≤ 5 с	US-1
FR-002	Користувач може увійти в застосунок	US-1	Must	Вхід успішний після правильних даних	US-1
FR-003	Користувач може вибрати одну з трьох кав'ярень	UC-2	Must	Показано всі три кав'ярні	UC-2 (крок «вибір кав'ярні»)
FR-004	Система показує актуальне меню	UC-2	Must	Меню відкривається за ≤ 3 с	UC-2 (крок «перегляд меню»)
FR-005	Користувач може оформити замовлення	UC-2	Must	Замовлення створюється після підтвердження	UC-2 (основний сценарій)
FR-006	Користувач може оплатити замовлення онлайн	US-2	Must	Після оплати створюється замовлення	US-2, UC-2 (крок «Оплатити»)

FR-007	Система нараховує бонусні бали	US-3	Could	Бонуси додаються після завершення замовлення	US-3
FR-008	Система надсилає Push-сповіщення	US-4	Must	Сповіщення приходить за ≤ 30 с	US-4, UC-2 (статус «Готове»)
FR-009	Менеджер бачить усі замовлення	US-5	Must	Нові замовлення з'являються у списку	US-5
FR-010	Бариста змінює статус замовлення	US-4	Must	Статус успішно змінюється	US-4

Завдання 4. Нефункціональні вимоги

Категорія ISO 25010 / 29148	ID	Нефункціональна вимога з вимірюваною метрикою	Трасованість (Джерело)
Продуктивність (Performance Efficiency)	NFR-P-001	Час відгуку сервера (Server Response Time) при обробці запиту створення замовлення не повинен перевищувати 300 ms на 95-му перцентилі (P95).	UC-01, Бізнес-мета (зменшення черг)
Продуктивність (Performance Efficiency)	NFR-P-002	Pusher/Push-сповіщення про готовність кави повинно доставлятися на пристрій користувача протягом не більше 3 секунд після зміни статусу баристою.	UC-03, Вхідні вимоги замовника
Надійність (Reliability & Availability)	NFR-R-001	Доступність серверної частини (Uptime) має становити не менше 99.5%	Обмеження бізнесу

		щомісяця у робочі години кав'ярень (07:00–21:00).	
Безпека (Security)	NFR-S-001	Автентифікація користувачів здійснюється за допомогою JWT-токенів з терміном дії 24 години; передача даних проводиться виключно за протоколом HTTPS (TLS 1.3).	Стандарти безпеки даних
Зручність використання (Usability)	NFR-U-001	Інтерфейс мобільного додатка має бути адаптивним для екранів з роздільністю від 320px до 1920px (підтримка iOS та Android).	Обмеження замовника

Розділ «Технічні обмеження та стек технологій»

- **Склад команди розробки (2 особи):**

Розробник 1 (Full-stack / Mobile Lead): відповідає за мобільний додаток (iOS/Android) та UI/UX дизайн.

Розробник 2 (Backend & Integration Lead): відповідає за серверний API, базу даних, веб-кабінет баристи та інтеграцію з POS-системою кав'ярні.

- **Платформи мобільної розробки:**

Додаток розробляється як кросплатформений (Cross-platform) для одночасного запуску на **iOS (v14.0+)** та **Android (v9.0+)**, що дозволяє зекономити ресурси та час команди з 2 осіб.

- **Рекомендований стек технологій:**

Frontend / Mobile: Flutter (Dart) або React Native — дає змогу писати один код для двох платформ.

Backend API: Node.js (Express / NestJS) або Python (FastAPI) — легкі у розгортанні та швидкі для створення REST API.

База даних: PostgreSQL (або Firebase / Supabase) — для зберігання користувачів, меню, статусів замовлень та накопичувальних бонусів.

Веб-кабінет баристи: Простий SPA-додаток (React / Vue.js або вбудований у Flutter Web) для відображення черги замовлень у реальному часі.

- **Інтеграційні обмеження:**

Обов'язкова двостороння інтеграція через REST API з наявною POS-системою кав'ярні (наприклад, Poster чи iiko) для передачі сформованих замовлень і зчитування номенклатури/залишків.

Інтеграція з банківським еквайрингом (MonoPay, LiqPay або Apple/Google Pay SDK) для приймання безготівкової оплати.

- **Часові обмеження (MVP):**

Жорсткий дедлайн релізу — **до 1 грудня**. Через обсяг команди (2 особи) пріоритет віддається реалізації лише базованих функцій (Must Have), а складні додаткові модулі переносяться на наступні ітерації.

Завдання 5. Критерії прийнятності та Definition of Done

1. Формальні критерії прийнятності (Given-When-Then сценарії)

- **Сценарій 1: Попереднє замовлення кави без черги (FR-001)**

Given (Дано): Авторизований користувач знаходиться в додатку та вибрав кав'ярню «CoffeeGo» на вул. Центральній.

When (Коли): Користувач обирає «Лате 300мл», додає до кошика, вказує час забрання «через 15 хв» та натискає «Оплатити».

Then (Тоді): Система успішно списує кошти через платіжний шлюз, створює замовлення зі статусом «Прийнято» та передає його в веб-кабінет баристи протягом < 2 секунд.

- **Сценарій 2: Нарахування бонусних балів (FR-002)**

Given (Дано): Користувач має баланс 50 бонусних балів і здійснює покупку на суму 100 грн.

When (Коли): Транзакція успішно підтверджується еквайрингом.

Then (Тоді): Система нараховує 10% від суми покупки (10 балів), і новий баланс користувача стає 60 балів протягом 5 секунд після оплати.

- **Сценарій 3: Отримання сповіщення про готовність (FR-003)**

Given (Дано): Замовлення користувача перебуває в статусі «Готується».

When (Коли): Бариста в веб-кабінеті натискає кнопку «Замовлення готове».

Then (Тоді): На телефон користувача протягом 3 секунд приходить Push-сповіщення: «Ваша кава готова! Заберіть на стійці видачі», а статус змінюється на «Очікує видачі».

2. Чеклист Definition of Done (DoD) для релізу

1. **Реалізація функціоналу:** Усі вимоги з пріоритетом Must Have повністю реалізовані в коді та пройшли перевірку Code Review.
2. **Тестування:** Усі позитивні та негативні тестові сценарії (Manual & Automated QA) пройдені, відсутні критичні баги (Blocker/Critical bugs = 0).
3. **Кросплатформеність:** Додаток протестований та стабільно працює на реальних пристроях Android та iOS з різними роздільностями екрана.
4. **Інтеграція:** Успішно проведено інтеграційне тестування з наявною POS-системою кав'ярні та платіжним шлюзом.
5. **Документація:** Підготовлено інструкцію користувача для баристи/менеджера щодо роботи з веб-кабінетом замовлень.
6. **Розгортання (Deployment):** Складено та протестовано продуктивну збірку, додаток опубліковано в App Store / Google Play (або передано на бета-тестування).

Завдання 6. Порівняльна вправа: ГОСТ $\rightleftharpoons$ SRS

Оскільки наш документ розроблявся за стандартом ISO, виконуємо конвертацію двох вимог у формат застарілого ГОСТ 34.602-89.

1. Переформулювання двох вимог у структурі стандарту ГОСТ 34.602-89

- **Вимога 1 (FR-001 — Замовлення кави):**

Переформулювання для ГОСТ (Розділ 3.1.1 «Вимоги до підсистем/функцій»):
«Система повинна забезпечувати можливість віддаленого формування замовлення напоїв та супутніх товарів користувачем із зазначенням бажаного часу видачі та автоматичною передачею даних до POS-системи кав'ярні».

- **Вимога 2 (FR-003 — Сповіщення про готовність):**

Переформулювання для ГОСТ (Розділ 3.1.2 «Вимоги до інформаційного обміну»): «Інформаційна підсистема сповіщень повинна здійснювати автоматичну генерацію та відправку Push-повідомлень на мобільний термінал користувача при зміні стану замовлення в базі даних на значення "Завершено"».

2. Рефлексія (порівняння підходів)

При переході від SRS (ISO 29148) до ГОСТ 34.602-89 формулювання вимог стали більш канцеляризованими, узагальненими та орієнтованими на описову структуру підсистем. У форматі ГОСТ бракує явного зв'язку з джерелами вимог (трасованості до Use Cases) та гнучкого опису дій користувача через очікувані результати. Для комерційного Agile-проєкту **«CoffeeGo»** з обмеженою командою (4 особи) та стислими термінами (до 1 грудня) значно зручнішим є **стандарт ISO 29148 / SRS**. Він дозволяє чітко розбити вимоги на атомарні користувацькі сценарії, додати конкретні вимірювані метрики прийнятності та забезпечити швидку комунікацію між розробниками без зайвої бюрократії.

Таблиця змін документів

Версія	Дата	Автори	Опис змін
1.0	09.09.2026	Баль М., Сівук М.	Створено першу версію технічного завдання проєкту CoffeeGo. Додано вибір стандарту SRS, загальний опис системи, функціональні та нефункціональні вимоги, критерії прийнятності, Definition of Done і порівняльну вправу.

Контрольні запитання

1. ГОСТ 34.602-89 орієнтований на каскадну модель розробки (Waterfall), юридичну суворість та великі державні/корпоративні системи із жорсткою бюрократією. SRS (ISO 29148) орієнтований на опис продуктів, зрозумілий як замовнику, так і команді розробників, ідеально підходить для Agile/Scrum, підтримує атомарність вимог та їх трасованість (Traceability) до тестів і сценаріїв.

2. Критерій прийнятності (Acceptance Criteria) стосується конкретної окремої вимоги або User Story (перевіряє, що саме має робити конкретна функція, наприклад, через Given-When-Then). Definition of Done (DoD) стосується всього

релізу або інкременту продукту і визначає загальні стандарти якості (проходження код-рев'ю, відсутність багів, наявність документації, успішне деплоєння).

3. Без вимірюваних метрик (наприклад, «час відгуку < 300 ms» замість «система має працювати швидко») неможливо об'єктивно перевірити виконання вимоги під час тестування. Метрики усувають суб'єктивність та розмитість формулювань між замовником і розробниками.

4. Трасованість — це можливість простежити шлях кожної вимоги від її джерела (User Story, бізнес-потреби) до конкретних елементів архітектури, коду та тест-кейсів. Вона потрібна для того, щоб переконатися, що жодна вимога не була втрачена або реалізована даремно без бізнес-потреби.